

Cost-Constrained Sequential Acquisition of Complete Data Batches

Abstract

We study the sequential acquisition of complete supervised-learning data. Each acquisition obtains a batch containing both predictor variables and target values. Data acquisition, model retraining, and model evaluation consume separate resources. The system must therefore decide whether to acquire another batch, retrain the predictive model, evaluate the current model, or stop. Retraining changes both predictive performance and the utility assigned to future data acquisitions. We formulate this problem as a constrained Markov decision process and use Q-learning to learn a sequential policy over the four actions. We consider two treatments of resource constraints: hard budget constraints enforced through action masking, and expected constraints handled through a Lagrangian objective.

1 Problem Setting

Let D_0^{train} denote an initially available labeled training dataset. Let

$$\mathcal{B}_0 = \{B_1, \dots, B_M\}$$

denote a collection of candidate data batches. Each batch contains complete supervised observations:

$$B_j = (X_j, Y_j),$$

where X_j contains predictor variables and Y_j contains the corresponding target values.

Before acquisition, the complete contents of a batch are unavailable to the learning system. The system may observe a low-cost preview or descriptor z_j , such as source information, batch size, acquisition price, collection period, coarse feature summaries, or an unlabeled preview sample. Acquiring a batch reveals both X_j and Y_j .

The system must sequentially decide:

- (i) whether to acquire another batch;
- (ii) when to retrain the model;
- (iii) when to evaluate the model; and
- (iv) when to stop.

2 Experimental Dataset

The first experiment uses the MiniBooNE Particle Identification dataset from the UCI Machine Learning Repository. The dataset contains 130,065 observations, 50 continuous predictor variables, and a binary target distinguishing signal events from background events [1].

The data are partitioned into:

- an initially available labeled training set;
- an acquisition dataset divided into candidate batches;

- an evaluation set;
- a hidden validation set; and
- a held-out test set.

The initial training size, candidate-batch size, number of candidate batches, and split proportions are experimental parameters. The first prototype uses 100 initial training records and candidate batches of 50 records.

3 Predictive Model

The predictive model is a histogram-based gradient-boosted tree classifier. For an observation x , the model estimates

$$f_{\theta_t}(x) = P_{\theta_t}(Y = 1 \mid x),$$

where θ_t denotes the current model parameters.

Histogram-based gradient boosting is used because it captures nonlinear relationships, produces class-probability estimates, trains efficiently on medium-sized tabular datasets, and supports repeated retraining during sequential experiments. Model hyperparameters are left to experimental selection.

4 Batch Acquisition Utility

Each candidate batch B_j has a visible descriptor or preview z_j . The current predictive model assigns an acquisition utility

$$q_t(B_j) = g(z_j; \theta_t, m_t),$$

where m_t contains the current acquisition and evaluation diagnostics.

A concrete first implementation uses an unlabeled preview sample $P_j \subset X_j$ and computes mean predictive entropy:

$$q_t(B_j) = \frac{1}{|P_j|} \sum_{x \in P_j} H(P_{\theta_t}(Y \mid x)).$$

For binary classification,

$$H(P_{\theta_t}(Y \mid x)) = -p_t(x) \log p_t(x) - (1 - p_t(x)) \log(1 - p_t(x)),$$

where $p_t(x) = P_{\theta_t}(Y = 1 \mid x)$.

The candidate batch with the highest feasible utility is selected:

$$B_t^* = \arg \max_{B_j \in \mathcal{B}_t} q_t(B_j).$$

Because the utility depends on θ_t , retraining changes the ranking of future candidate batches.

5 State

At decision step t , the state is

$$s_t = \left(D_t^{\text{train}}, D_t^{\text{pending}}, \mathcal{B}_t, \theta_t, m_t, b_t \right).$$

For tabular Q-learning, this state is represented using a compact discretized feature vector containing:

- current training-set size;
- pending-data size;
- number of remaining candidate batches;
- remaining acquisition, retraining, and evaluation budgets;
- most recent evaluation score;
- acquisitions since the last retraining;
- retrainings since the last evaluation; and
- mean and maximum candidate-batch utility.

Continuous variables are discretized into finite intervals.

6 Actions

The action space is

$$\mathcal{A} = \{\text{ACQUIRE}, \text{RETRAIN}, \text{EVALUATE}, \text{STOP}\}.$$

6.1 Acquire

The feasible candidate batch with the highest acquisition utility is selected:

$$B_t^* = \arg \max_{B_j \in \mathcal{B}_t} q_t(B_j).$$

The complete batch is added to pending data and removed from the candidate collection:

$$D_{t+1}^{\text{pending}} = D_t^{\text{pending}} \cup B_t^*, \quad \mathcal{B}_{t+1} = \mathcal{B}_t \setminus \{B_t^*\}.$$

The model is unchanged:

$$\theta_{t+1} = \theta_t.$$

6.2 Retrain

All pending data are incorporated:

$$D_{t+1}^{\text{train}} = D_t^{\text{train}} \cup D_t^{\text{pending}}, \quad D_{t+1}^{\text{pending}} = \emptyset.$$

The predictive model is refitted:

$$\theta_{t+1} = \text{Train}(D_{t+1}^{\text{train}}).$$

Retraining changes predictive performance and future acquisition utilities, but it does not automatically reveal model performance.

6.3 Evaluate

The current model is evaluated on the designated evaluation set. The resulting performance statistic is added to m_t and becomes visible to the decision policy. Evaluation changes policy information but does not modify the model or training data.

6.4 Stop

The episode terminates and returns the current predictive model. Pending data that have not been incorporated through Retrain do not affect the terminal model.

7 Sequential Decision Formulation

The problem is formulated as a Markov decision process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma),$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s_{t+1} | s_t, a_t)$ is the transition process, r_t is the reward, and γ is the discount factor. The policy directly chooses one of the four actions at every decision step:

$$a_t = \pi(s_t).$$

8 Q-Learning

Q-learning estimates the long-term value $Q(s, a)$ of taking action a in state s . After observing a transition (s_t, a_t, r_t, s_{t+1}) , the corresponding value is updated as

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right],$$

where α is the learning rate and γ controls the importance of future utility.

The learned policy is

$$\pi(s) = \arg \max_a Q(s, a).$$

During training, exploration is introduced using an epsilon-greedy policy.

9 Reward

The primary reward is terminal predictive utility. For nonterminal actions,

$$r_t = 0.$$

When Stop is selected,

$$r_T = U(\theta_T),$$

where $U(\theta_T)$ is measured on the hidden validation set during policy development. This sparse reward allows Q-learning to propagate terminal model utility backward to earlier decisions.

A small step penalty may optionally discourage unnecessarily long episodes, but acquisition, retraining, and evaluation costs are treated separately through constraints.

10 Cost Model

The cumulative costs are

$$C_T^{\text{acq}} = \sum_{t < T} c_{\text{acq}}(a_t, s_t), \quad C_T^{\text{train}} = \sum_{t < T} c_{\text{train}}(a_t, s_t),$$

and

$$C_T^{\text{eval}} = \sum_{t < T} c_{\text{eval}}(a_t, s_t).$$

Acquisition cost may depend on batch size, source, purchase price, or collection difficulty. Retraining cost may depend on training-set size, elapsed time, memory use, processor use, or energy consumption. Evaluation cost may depend on evaluation-set size, delayed outcome collection, expert review, or computational effort.

11 Approach I: Hard Budgets with Action Masking

Let the initial budgets be

$$B_{\text{acq}}, \quad B_{\text{train}}, \quad B_{\text{eval}}.$$

The remaining budgets are included in the state:

$$b_t = \left(b_t^{\text{acq}}, b_t^{\text{train}}, b_t^{\text{eval}} \right).$$

Each resource-consuming action subtracts its cost from its corresponding budget.

At each step, define the feasible action set:

$$\mathcal{A}_{\text{feasible}}(s_t) = \{a \in \mathcal{A} : c(a, s_t) \leq b_t\}.$$

Q-learning selects only among feasible actions:

$$a_t = \arg \max_{a \in \mathcal{A}_{\text{feasible}}(s_t)} Q(s_t, a).$$

Infeasible actions are masked and cannot be selected.

The optimization problem is

$$\max_{\pi} \mathbb{E}_{\pi}[U(\theta_T)]$$

subject to

$$C_T^{\text{acq}} \leq B_{\text{acq}}, \quad C_T^{\text{train}} \leq B_{\text{train}}, \quad C_T^{\text{eval}} \leq B_{\text{eval}}.$$

This approach guarantees exact budget compliance. The first prototype uses unit action costs and hard-budget action masking.

12 Approach II: Lagrangian Constrained Q-Learning

The second approach handles expected resource constraints using nonnegative Lagrange multipliers $\lambda_{\text{acq}}, \lambda_{\text{train}}, \lambda_{\text{eval}}$. The constrained objective is

$$\max_{\pi} \mathbb{E}_{\pi}[U(\theta_T)]$$

subject to expected cost constraints. The modified reward is

$$r_t^{\lambda} = r_t - \lambda_{\text{acq}} c_t^{\text{acq}} - \lambda_{\text{train}} c_t^{\text{train}} - \lambda_{\text{eval}} c_t^{\text{eval}}.$$

Q-learning then uses r_t^{λ} .

The multipliers are updated according to observed budget violations. For example,

$$\lambda_{\text{acq}} \leftarrow \left[\lambda_{\text{acq}} + \eta_{\lambda} (C_T^{\text{acq}} - B_{\text{acq}}) \right]_+,$$

with analogous updates for retraining and evaluation, where $[x]_+ = \max(x, 0)$.

This approach learns a predictive-utility-cost trade-off, but it does not guarantee that every individual episode satisfies the budgets.

13 Comparison of Budget Approaches

Hard-budget action masking should be used when no episode may exceed the resource limits, costs are known before execution, and compliance must be guaranteed.

Lagrangian constrained Q-learning should be used when constraints apply on average, costs may be uncertain, occasional violations are acceptable, and the policy should learn the utility–cost trade-off.

The first experiment uses hard-budget action masking as the primary formulation. Lagrangian constrained Q-learning is reserved for a second experiment.

14 Experimental Comparison

The sequential Q-learning policy should be compared against:

- retraining after every acquisition;
- retraining after a fixed number of acquisitions;
- evaluation after every retraining;
- fixed evaluation intervals;
- one final retraining;
- random feasible action selection;
- hard-budget Q-learning; and
- Lagrangian constrained Q-learning.

Reported metrics should include final test accuracy, final test AUROC, all three cumulative costs, numbers of acquisitions, retrainings, and evaluations, terminal predictive utility, and budget-violation frequency.

15 Central Research Question

How should a learning system sequentially choose between acquiring a complete data batch, retraining the model, evaluating the model, and stopping, in order to maximize terminal predictive utility under acquisition, retraining, and evaluation constraints?

References

- [1] UCI Machine Learning Repository. MiniBooNE Particle Identification Dataset. <https://archive.ics.uci.edu/dataset/199/miniboone+particle+identification>.